Group 37 Project Step 2 Draft – Coffee Shop Inventory and Sales Management System

Team Members: Carl Curran, Thien Ngan Nguyen

Feedback By The TAs and Peer Reviewer

1. Timothy Sorisa

“Hello, Group 37.
The overview has a clear use case and purpose for the application. I also like that you go into a realistic business need on why tracking this stuff would be important to a coffee shop. Is tracking these things important for operation, but also for cost savings. That is a great argument for implementing a system, because let's face it, at the end of the day it's about the dollar. The statistics add real depth into the problem your application can help address and mitigate. The statistics also made reading the overview more engaging.
The five entities presented each are clear and sensible. I can easily see how they relate to a coffee shop and its operation. Each of the entities have sensible attributes. I would consider having description attributes NULLable, because I don’t think that they are essential to how the tables function. There are many M:M relationships. I think this adds a lot of complexity, due to needing junction tables for these relationships. One design change I could see would maybe be removing Sales as an entity and just having a # of sold attributes. That could keep the focus on the supply chain.
The diagram has all entities, but I believe it is missing the M:M junction tables. I think a good name for the junction table between Items and Ingredients would be recipes. Disregarding junction tables, the ER diagram is logical; relationships are clearly shown. The styling is consistent through the diagram.
Citation:
Date: 10/12/2026
Prompts used to clarify table relationships
With SQL databases when tables have a many to many relationship do they need another table in between.
ChatGPT. (October 2026). OpenAI ChatGPT [LLM]. https://chatgpt.com/”

2. Kyle Parker

“Hello Group 37. I think it is cool that you found sources to back up the importance of your database to the small coffee shop.

- Overview: Your database is designed for use at a small coffee shop. Your database will have a website user interface. The database will track sales of items, supply of ingredients, vendors and incoming invoices from said vendors. You also do a good job of noting the importance of a database solution for a coffee shop with sources to back it up.
- Specificity: Your specificity is very good, you have distinct numbers for what the database will need to handle, 300 average daily sales, 20 menu items to track sales for, 30 ingredients to manage stock for and 5 vendors to purchase stock from.
- Entities: You describe 5 entities, which exceeds the assignment requirements without going overboard. They also correctly describe a single idea, that is **Sales** which records individual transaction information, **Items** which are the different items on your menu, **Ingredients** which make up your menu items, **Invoices** from vendors to purchase ingredients needed to make your menu items, and **Vendors** from whom you purchase ingredients and receive invoices.
- Entity Details: Your database outline details the purpose of each entity well, and also describes relationships. Your outline also lists attribute datatype and constraints aptly. The only thing I notice is that you do not list what attributes will be used as a Foreign Key in the junction tables for your 3 M:N relationships.
- ERD: Your ERD is well organized and displayed but is missing one aspect. You only list participation in one of your connections (between Invoices and Vendors), and only on a single side of the relationship.
- Naming Consistency: The naming for your entities and attributes is consistent. Your entities start with a capital and are plural, while your attributes are in CamelCase starting in lowercase and singular.

Feedback was all my own work.”

3. Kiran Desimone

“Your draft describes a database-driven website that lets a small coffee shop track sales, ingredients, inventory, vendors, and purchase invoices in one place. Its main goals are to review item sales, monitor stock, flag ingredients nearing their reorder level, and track incoming purchases. I see the problem the DBMS is trying to solve but I think the overview would benefit from something more specific such as "The shop currently tracks stock on paper and in a spreadsheet, so baristas run out of oat milk mid-shift and the manager can't tell which items are most profitable."

I count 5 entities: Items, Ingredients, Vendors, Invoices, and Sales. All of these entities make sense for a coffee shop. However, Ingredients also contains supplies, so maybe separate the two entities or rename Ingredients to IngredientItems.
I would say the outline mostly describes the relationships between entities. I know intersection tables were not supposed to be included in the ERD, but my team included them in the outline which I think helps see the overall structure. I would also suggest making things like itemName and vendorName unique so you don't have duplicates. Other than that I think everything else looks good and this is all my work."

4. Ruth Ortiz Barron

"Hi Group 37! Your database idea is great. The entities listed are clear and the relationships are explained well. One thing that I noticed was that FKs were not directly identified in your outline and in your diagram. I think that would make the relationship easier to follow without the explanations."

Actions Based On The Feedback

Based on Timothy's feedback, we changed the description attributes in Items and Ingredients from NOT NULL to NULL because descriptions are optional information and are not required for those records to function.
We also added intersection tables for the three M:N relationships in our original design. Sales and Items are now connected through SalesItems, Items and Ingredients are connected through ItemIngredients, Invoices and Ingredients are connected through InvoiceIngredients. These intersection tables include the foreign keys from the two entities they connect.
We kept Sales as an entity instead of replacing it with only a number sold attribute. Sales represents each individual transaction and stores information such as the date and time of the transaction. Keeping individual Sales also lets the system connect multiple Items to a single Sale through SalesItems. Using only a number sold attribute would not preserve individual transaction history.
We did not use the name recipes for the intersection table between Items and Ingredients. We used ItemIngredients instead because the name directly identifies the two entities being connected and follows the naming style used by the other intersection tables.
Based on Kyle's feedback, we directly identified all foreign keys in the database outline and the updated ERD. We also updated the ERD so participation is shown on both sides of every relationship.
Based on Kiran's feedback, we made the problem in the overview more specific by explaining why keeping sales, ingredient inventory and vendor purchase information

connected is useful to the coffee shop. We also changed the Ingredients entity so it represents ingredients only instead of combining ingredients and general supplies in the same entity.
We made itemName and vendorName UNIQUE to help prevent duplicate records.
Based on Ruth's feedback, all foreign keys are now directly marked with FK in both the database outline and the ERD instead of relying only on relationship explanations.
Based on the feedback from our TA Praseeda we were advised to update the overview to describe a Python class that interacts with the database instead of a database-driven website. Also, we need to make sure we export the ERD at a higher resolution so the field names are easier to read.

Upgrades To The Draft Version

In addition to the changes based on peer feedback, we made ingredientName UNIQUE so the same ingredient is less likely to be entered more than once.
We also added attributes to the intersection tables that describe information specific to those relationships. SalesItems records the quantity of an Item sold and its price at the time of the Sale. ItemIngredients records the amount of an Ingredient needed for an Item. InvoiceIngredients records the quantity purchased and unit cost of each Ingredient on an Invoice.The new intersection table names also follow the same naming style as the rest of the database.
For Step 2, we also added UNIQUE constraints to the pairs of foreign keys in our intersection tables. We checked our sample data so the sale and invoice totals match their detail records.

Overview

The Coffee Shop Inventory and Sales Management System is a Python class designed to interact with a Database, which we are making to help a small coffee shop manage item sales, ingredients, inventory purchases and vendor information in one place. Without one connected system, it can be difficult to see how item sales relate to ingredient usage and when ingredients need to be reordered. For the purpose of this project, the coffee shop is assumed to process approximately 300 sales per day, offer 20 menu items, maintain about 30 different ingredients and purchase inventory from 5 vendors. Inventory management is important for a cafe because poor inventory control can result in wasted ingredients, missing items and unnecessary costs, while digital inventory systems can help monitor stock and vendor orders (Crimson Cup, 2025). The importance of managing these costs is

also reflected in National Restaurant Association data which found that food and non-alcohol beverage costs represented a median 32.4% of sales among limited service restaurant respondents in 2024 (National Restaurant Association, 2025).

The database will record Sales completed by the coffee shop, Items included in each Sale, the Ingredients required for each Item, Vendors that supply those Ingredients and Invoices for inventory purchases from Vendors. Using this Python class will allow the coffee shop to review item sales, monitor current Ingredient inventory levels, identify ingredients that are approaching their reorder levels and keep track of incoming inventory purchases.

Database Outline

- Items: records the menu items sold by the coffee shop
    - itemID: int, auto_increment, unique, NOT NULL, PK
    - itemName: varchar(100), NOT NULL, UNIQUE
    - description: varchar(255), NULL
    - unitPrice: decimal(8,2), NOT NULL
    - isActive: boolean, NOT NULL
    - Relationship:
        - Items has a 1:M relationship with SalesItems. One Item may appear in zero or many SalesItems records while each SaleItems record belongs to exactly one Item
        - Items has a 1:M relationship with ItemIngredients. Each Item requires one or more ItemIngredients records while each ItemIngredients record belongs to exactly one Item
        - The relationship between Items and Sales is conceptually M:N and is implemented through SaleItems
        - The relationship between Items and Ingredients is conceptually M:N and is implemented through ItemIngredients
- Ingredients: stores the ingredients kept in inventory and used to make Items
    - ingredientID: int, auto_increment, unique, NOT NULL, PK
    - ingredientName: varchar(100), NOT NULL , UNIQUE
    - description: varchar(255), NULL
    - unit: varchar(20), NOT NULL
    - inventoryQty: decimal(10,2), NOT NULL
    - reorderLevel: decimal(10,2), NOT NULL
    - shelfLifeDays: int, NULL

    - Relationship:
        - Ingredients has a 1:M relationship with ItemIngredients. One Ingredient may appear in zero or many ItemIngredients while each ItemIngredients record references exactly one Ingredient
        - Ingredients has a 1:M relationship with InvoiceIngredients. One Ingredient may appear in zero or many InvoiceIngredients records while each InvoiceIngredients record references exactly one Ingredient
        - The relationship between Ingredients and Items is conceptually M:N and is implemented through ItemIngredients
        - The relationship between Ingredients and Invoices is coneptually M:N and is implemented through InvoiceIngredients

- Vendors: stores information about businesses that supply Ingredients to the coffee shop
    - vendorID: int, auto_increment, unique, NOT NULL, PK
    - vendorName: varchar(100), NOT NULL, UNIQUE
    - address: varchar(255), NULL
    - phone: varchar(20), NULL
    - contactEmail: varchar(100), NULL
    - Relationship:
        - Vendors has a 1:M relationship with Invoices. One Vendor may have zero or many Invoices while each Invoice belongs to exactly one Vendor. vendorID is an FK in Invoices.

- Invoices: records inventory purchases made from Vendors
    - invoiceID: int, auto_increment, unique, NOT NULL, PK
    - vendorID: int, NOT NULL, FK to Vendors.vendorID
    - invoiceDate: date, NOT NULL
    - totalCost: decimal(10,2), NOT NULL
    - Relationship:
        - Invoices has an M:1 relationship with Vendors. Each Invoice belongs to exactly one Vendor while one Vendor may have zero or many Invoices.
        - Invoices has a 1:M relationship with InvoiceIngredients. Each Invoice contains one or more InvoiceIngredients records while each InvoiceIngredients record belongs to exactly one Invoice

        - The relationship between Invoices and Ingredients is conceptually M:N and is implemented through InvoiceIngredients

- Sales: records individual sales transactions completed by the coffee shop
    - saleID: int, auto_increment, unique, NOT NULL, PK
    - saleDateTime: datetime, NOT NULL
    - totalSaleValue: decimal(10,2), NOT NULL
    - Relationship:
        - Sales has a 1:M relationship with SalesItems. Each Sale contains one or more SalesItems records while each SalesItem record belongs to exactly one Sale
        - The relationship between Sales and Items is conceptually M:N and is implemented through SalesItems

- SalesItems: intersection table that connects Sales and Items and records the Item included in each Sale
    - saleItemID: int, NOT NULL, PK, unique, auto_increment
    - saleID: int, NOT NULL, FK
    - itemID: int, NOT NULL, FK
    - quantity: int, NOT NULL
    - salePrice: decimal(8,2), NOT NULL
        - SalesItems has a M:1 relationship with Sales. Each SalesItems record belongs to exactly one Sale while one Sale contains one or more SalesItems records. FK to Sales.saleID
        - SalesItems has a M:1 relationship with Items. Each SalesItems record references exactly one Item while one Item may appear in zero or many SalesItems records. FK to Items.itemID
        - UNIQUE (saleID, itemID) prevents the same Item from appearing twice on one Sale

- ItemIngredients: intersection table that connects Items and Ingredients and records the amount of each Ingredient required for an Item
    - itemIngredientID: int, NOT NULL, PK, unique, auto_increment
    - itemID: int, NOT NULL, FK
    - ingredientID: int, NOT NULL, FK
    - quantityRequired: decimal(10,2), NOT NULL

        - ItemIngredients has a M:1 relationship with Items. Each ItemIngredients record belongs to exactly one Item while each Item has one or more ItemIngredients records, FK to Items.itemID
        - ItemIngredients has a M:1 relationship with Ingredients. Each ItemIngredients record references exactly one Ingredient while one Ingredient may appear in zero or many ItemIngredients records. FK to Ingredients.ingredientID
        - UNIQUE (itemID, ingredientID) prevent duplicate Ingredient entries for one Item

- InvoiceIngredients: intersection table that connects Invoices and Ingredients and records the Ingredients purchased on each Invoice
    - invoiceIngredientID: int, NOT NULL, PK, unique, auto_increment
    - invoiceID: int, NOT NULL, FK
    - ingredientID: int, NOT NULL, FK
    - quantityPurchased: decimal(10,2), NOT NULL
    - unitCost: decimal(10,2), NOT NULL
    - InvoiceIngredients has a M:1 relationship with Invoices. Each InvoiceIngredients record belongs to exactly one Invoice while each Invoice contains one or more InvoiceIngredients records. FK to Invoices.invoiceID
    - InvoiceIngredients has a M:1 relationship with Ingredients. Each InvoiceIngredients record references exactly one Ingredient while one Ingredient may appear in zero or many InvoiceIngredients records. FK to Ingredients.ingredientID
    - UNIQUE (invoiceID, ingredientID) prevents duplicate Ingredient entries on one Invoice

Entity-Relationship Diagram

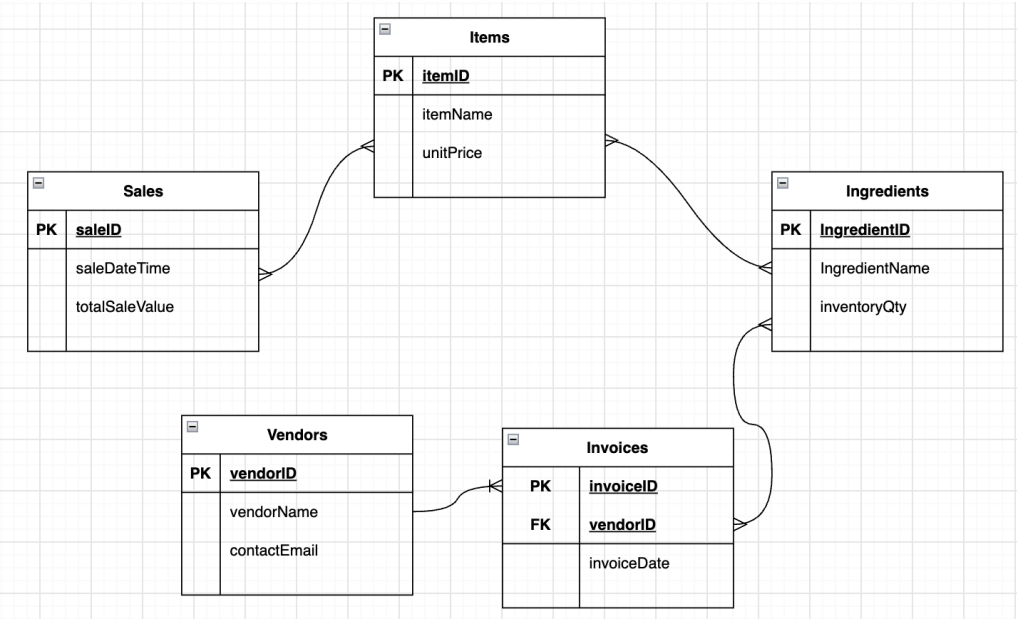


Figure 1. Conceptual ERD showing the five main entities

Our ERD shows how the coffee shop records Sales, Items, Ingredients, Vendors and Invoices. Sales and Items have an M:N relationship because a Sale can include multiple Items and an Item can appear in many Sales. Items and Ingredients also have an M:N relationship because one Item can use several Ingredients and an Ingredient can be used in different Items. Invoices and Ingredients have an M:N relationship because an Invoices can include several Ingredients and an Ingredient can be purchased on different Invoices. Vendors and Invoices have a 1:M relationship because one Vendor can supply many Invoices while each Invoice belongs to one Vendor. The intersection tables used to implement these relationships are shown in the schema diagram.

Schema Diagram

Our schema diagram shows all eight tables with their columns, data types, primary keys and foreign keys. SalesItems, ItemIngredients and InvoiceIngredients connect the three M:N relationships. Each has its own auto increment primary key and two foreign keys.

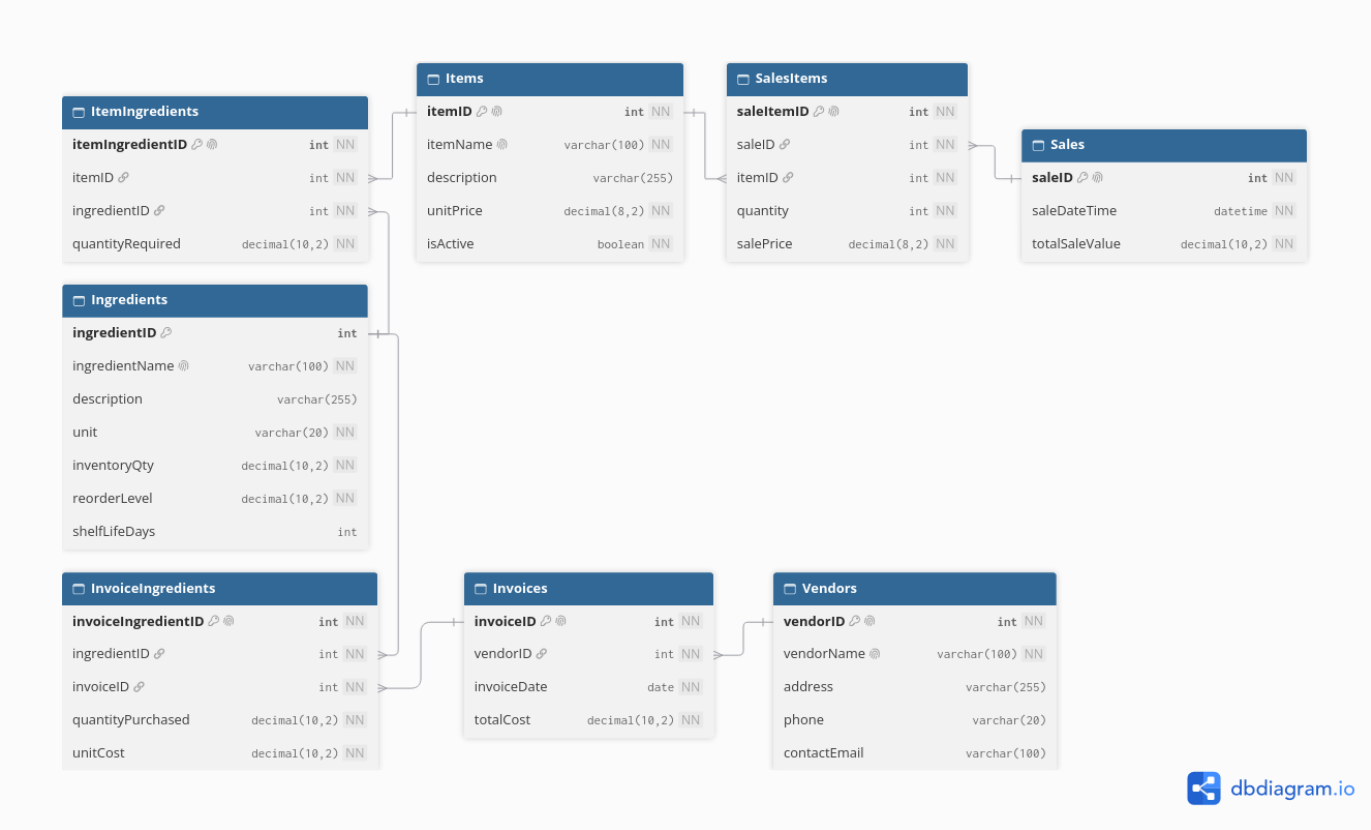


Figure 2. Complete schema diagram showing all eight tables.



Figure 2a. Enlarged schema detail (left side) to make the attributes easier to read.



Figure 2b. Enlarged schema detail (right side) to make the attributes easier to read.

Normalization Explanation

- Before normalization: If we kept a Sale in one row with a list of Items, we might write something like Sale 1 with Latte and Espresso, quantities of 2 and 1 and prices of 4.50 and 3.00. This would put several values in one field and make it harder to find or change one Item. An invoice report could also repeat the Vendor name and contact information for every Ingredient purchased. If that Vendor changed its phone number, we might need to update several records. We separated this information into different tables to avoid those problems.

- First Normal Form (1NF): Our schema meets 1NF because each table has a primary key and each field stores one value. For example, we do not keep a list of menu Items in the Sales table. We use SalesItems to record each Item included in a Sale. We also use ItemIngredients to record each Ingredient needed for an Item and InvoiceIngredients to record each Ingredient on an Invoice. This lets us store multiple related records without putting several values in one field.
- Second Normal Form (2NF): After checking 1NF, we looked for attributes that depended on only part of a key. We kept itemName and unitPrice in Items because those details belong to the menu Item, not to a particular Sale. SalesItems only records the Sale, Item, quantity and price when the Item was sold. We did the same for ItemIngredients and InvoiceIngredients. Ingredient details are kept in Ingredients while the intersection tables only store information about their relationships. We also kept each foreign key pair unique so the same Item or Ingredient is not repeated on one Sale, recipe or Invoice. These choices help avoid partial dependencies so our design meets 2NF.
- The UNIQUE constraints prevent duplicate relationship rows. Separating item and ingredient details into their own tables is what helps avoid partial dependencies.
- Third Normal Form (3NF): For 3NF, we checked if one non-key attribute determined another non-key attribute. For example, each Invoice belongs to a Vendor. If we also stored vendorName and contactEmail in Invoices, we would repeat the same information on many Invoices. Instead, we keep those details in Vendors and store vendorID as a foreign key in Invoices. We also keep menu details in Items and ingredient details in Ingredients instead of repeating them in sales or purchase records. We did not find any other transitive dependencies that would cause problems with our current design.
- We also kept totalSaleValue in Sales and totalCost in Invoices because they summarize each transaction. These totals must be updated when the detail rows change. The python class should calculate or update them so they stay consistent with SalesItems and InvoiceIngredients.

Example Data

The following tables show the example data we used to test our design. These tables use the same values as the INSERT statements in DDL.sql. The added records show that Item 4 has a recipe and Invoice 3 has an Ingredient purchase.

Items:

itemID	itemName	description	unitPrice	isActive
1	coffee	Medium blend drip coffee	2.50	True
2	cookie	Chocolate chip cookie	4.00	True
3	latte	1/3 espresso, 1/3 steamed milk, 1/3 foam	5.50	True
4	Pumpkin Spice Latte	Seasonal Fall Drink	6.50	False

Ingredients:

ingredientID	ingredientName	description	unit	inventoryQty	reorderLevel	shelfLifeDays
1	coffeebeans	Costa Rican Tarrazu city blend	lb	20	4	30
2	sugar	Raw Hawaiian Sugar	lb	5	1	240
3	milk	Whole Milk	gallon	3	1	7

Vendors:

vendorID	vendorName	address	phone	contactEmail
1	Costco	4401 4th Ave S, Seattle, WA 98134	(206) 622-3136	NULL
2	Prestogeorge Coffee & Tea	1719 Penn Ave. Pittsburgh, PA 15222	412-471-0133	customerservice@prestogeorge.com
3	Levain Bakery	1484 3rd Ave, New York, NY 10028	+19174645578	Lavain.contact@gmail.com

Invoices:

invoiceID	vendorID	invoiceDate	totalCost
1	1	2026-10-04	125.00
2	1	2023-05-03	400.00
3	3	2026-10-20	120

Sales:

saleID	saleDateTime	totalSaleValue
1	2026-10-21 10:05:00	13.50
2	2026-10-22 11:25:00	3.50
3	2026-10-23 09:15:00	17.00

SalesItems:

saleItemID	saleID	itemID	quantity	salePrice
1	1	2	1	5.50
2	1	1	2	4.00
3	2	3	1	3.50
4	3	3	2	5.50
5	3	1	2	3.00

Sale 1 contains Items 2 and 1 and Item 3 appears in Sales 2 and 3. This shows both sides of the M:N relationship. The salePrice value stores the selling price at the time of the Sale so it does not have to match the current Item.unitPrice

ItemIngredients:

itemIngredientID	itemID	ingredientID	quantityRequired
1	1	1	0.03
2	1	2	0.25
3	2	3	0.24
4	3	3	0.02
5	3	2	0.04
6	4	3	0.25

Item 1 has more than one Ingredient and Ingredient 3 is used by more than one Item. Item 4 now also has a recipe row.

InvoiceIngredients:

invoiceIngredientID	invoiceID	ingredientID	quantityPurchased	unitCost
1	1	1	10.00	10.00
2	1	2	5.00	5.00
3	2	3	20.00	20.00
4	3	2	40.00	3.00

Invoice 1 includes two Ingredients and Ingredient 2 appears in Invoice 1 and Invoice 3. The new row for Invoice 3 contains 40.00 units of sugar at 3.00 per unit which gives a total of 120.00

Citations

Crimson Cup (2025, October 29). Inventory tips to keep your cafe running smoothly. https://www.crimsoncup.com/whats-new/inventory-tips-to-keep-your-cafe-running-smoothly

National Restaurant Association. (2025, September 10). Restaurant operators kept food costs ratios in check in 2024. https://restaurant.org/research-and-media/research/restaurant-economic-insights/analysis-commentary/restaurant-operators-kept-food-cost-ratios-in-check-in-2024/

We used Grammarly to check grammar and spelling errors in our project. Grammarly. (n.d.). https://www.grammarly.com/